01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-01 · SOURCE PAGE 001/100

Workspace isolation

TeamFlow | Fictional project-and-task management SaaS | Documentation corpus | 27 October 2026

Allocation: 35 Business / 25 Functional / 15 API / 10 ADR / 10 Guidelines / 5 FAQ. Stable requirement IDs support ticket evidence references.

Policy intent: Workspaces are the top-level security and ownership boundary.

Normative business rules

BR-01-R01 A workspace has a unique immutable ID, name of 3-80 characters, billing owner and creation timestamp.

BR-01-R02 Every project, task, sprint, file, invitation and notification belongs to exactly one workspace.

BR-01-R03 A user may belong to multiple workspaces, but data from one workspace is never included in another workspace query.

BR-01-R04 Creating a workspace gives its creator the Workspace Owner role in that workspace.

BR-01-R05 Workspace URLs use opaque IDs; a workspace name change does not invalidate existing resource references.

BR-01-R06 Switching the active workspace changes the query scope but does not change the user identity.

Worked business scenario

A member of two workspaces searches for the same task title; each search returns only objects from its explicitly selected workspace.

Boundary and rejection example

A client sending an object ID from a different workspace receives NOT_FOUND rather than another tenant's data.

Test and implementation interpretation

Use BR-01-R01 through BR-01-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-02 · SOURCE PAGE 002/100

Account registration and invitation

Policy intent: Account lifecycle and invited membership are distinct concepts.

Normative business rules

BR-02-R01 Accounts use a verified email as their unique login identifier after trimming whitespace and case folding.

BR-02-R02 Users are provisioned by accepting a workspace invitation; self-registration alone does not grant project access.

BR-02-R03 Invitation links are single-use and expire seven calendar days after creation.

BR-02-R04 Invited email must match the email verified on the account that accepts the invitation.

BR-02-R05 Accepting an invitation creates one membership; a repeated acceptance cannot create a duplicate.

BR-02-R06 An invitation cancelled by its issuer cannot be accepted even if the original URL remains available.

Worked business scenario

A developer creates an account, verifies email and accepts an invitation to join a product team; no projects are visible beforehand.

Boundary and rejection example

Expired and cancelled invitations return separate machine-readable error codes and grant no membership.

Test and implementation interpretation

Use BR-02-R01 through BR-02-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-03 · SOURCE PAGE 003/100

Authentication and sessions

Policy intent: Authentication is required before any nonpublic workspace operation.

Normative business rules

BR-03-R01 Passwords are never returned by an API and are stored using a suitable password hashing function.

BR-03-R02 Password login requires a verified account and returns a short-lived access token plus a rotating refresh token.

BR-03-R03 Five consecutive incorrect passwords on an account trigger a fifteen-minute login lockout.

BR-03-R04 A user can revoke all active sessions from account security settings.

BR-03-R05 Refresh tokens are bound to a session record; reuse of an already rotated refresh token revokes that session.

BR-03-R06 Interactive sessions are invalidated after 30 minutes of inactivity, measured on server-accepted activity.

Worked business scenario

A returning user with a valid session can open the board; once the inactivity threshold is exceeded, the server requires fresh authentication.

Boundary and rejection example

An account lockout never reveals whether the email exists to an unauthenticated caller.

Test and implementation interpretation

Use BR-03-R01 through BR-03-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Workspace roles and grants

Policy intent: Authorization is role-based and enforced server-side on every request.

Normative business rules

BR-04-R01 The workspace roles are OWNER, ADMIN, MEMBER and GUEST; the highest role is not inferred from project membership.

BR-04-R02 OWNER can transfer ownership, manage workspace settings and grant any role; at least one OWNER must remain.

BR-04-R03 ADMIN can manage invitations and projects but cannot transfer workspace ownership.

BR-04-R04 MEMBER can access only projects where they have explicit project membership.

BR-04-R05 GUEST is read-only in projects to which they are explicitly invited and cannot access workspace-wide reports.

BR-04-R06 Role changes take effect for subsequent authorized requests without requiring a user to reaccept invitations.

Worked business scenario

An administrator adds a new member and then grants project membership; the member is not able to read unrelated projects.

Boundary and rejection example

The last OWNER cannot remove their own OWNER grant unless another OWNER is already assigned.

Test and implementation interpretation

Use BR-04-R01 through BR-04-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Project setup and lifecycle

Policy intent: Projects organize work within one workspace.

Normative business rules

BR-05-R01 A project requires a unique workspace-local key of 2-10 uppercase letters and a display name of 3-100 characters.

BR-05-R02 Project states are DRAFT, ACTIVE, ON_HOLD and ARCHIVED.

BR-05-R03 New projects start as DRAFT and require at least one Project Manager before activation.

BR-05-R04 Only project managers and workspace admins may put ACTIVE projects ON_HOLD or resume them.

BR-05-R05 ARCHIVED projects reject all task and sprint mutations while preserving historical reads for authorized users.

BR-05-R06 Project keys remain immutable after the first task is created, avoiding broken ticket identifiers.

Worked business scenario

An administrator creates project MOB and appoints a Project Manager before making it ACTIVE.

Boundary and rejection example

Attempting to create a second project with the key MOB inside the same workspace returns PROJECT_KEY_EXISTS.

Test and implementation interpretation

Use BR-05-R01 through BR-05-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-06 · SOURCE PAGE 006/100

Project membership and visibility

Policy intent: Workspace access does not imply access to every project.

Normative business rules

BR-06-R01 A project has OWNER-MANAGER, CONTRIBUTOR or OBSERVER project roles, each bound to a workspace member.

BR-06-R02 Project managers may edit settings, membership, sprint plans and task state; contributors may edit assigned work.

BR-06-R03 Observers can read project content but cannot modify tasks, comments or attachments.

BR-06-R04 Project membership is explicit; the creator becomes its initial Project Manager.

BR-06-R05 Projects with restricted visibility appear only to members, workspace ADMINS and OWNERs.

BR-06-R06 The member limit documented for an individual project is 75 active memberships, excluding deactivated accounts.

Worked business scenario

A workspace MEMBER invited to Project ALPHA cannot open a private Project BETA by guessing its ID.

Boundary and rejection example

When a member is removed from a project, task history still stores their authorship but current access is revoked.

Test and implementation interpretation

Use BR-06-R01 through BR-06-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Task identity and fields

Policy intent: Tasks are the primary unit of work with stable identifiers.

Normative business rules

BR-07-R01 Task types are STORY, BUG, TASK and EPIC; type is set on creation and can change only before sprint commitment.

BR-07-R02 A task has a stable key PROJECTKEY-NNNN, title of 5-140 characters and optional Markdown description.

BR-07-R03 STORY tasks require at least one acceptance criterion before entering READY.

BR-07-R04 BUG tasks require reproduction steps or an explicit reason why reproduction is unavailable.

BR-07-R05 Each task has exactly one parent project; moving a task across projects is not supported in the baseline.

BR-07-R06 Task creation records reporter, creator and timestamps separately; creator does not imply assignee.

Worked business scenario

A project manager creates MOB-0042 as a STORY with three acceptance criteria and links it to a sprint later.

Boundary and rejection example

Changing a task title must not change its key or invalidate its comment, audit and attachment references.

Test and implementation interpretation

Use BR-07-R01 through BR-07-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-08 · SOURCE PAGE 008/100

Task states and transitions

Policy intent: A task follows one controlled state machine.

Normative business rules

BR-08-R01 The allowed states are BACKLOG, READY, IN_PROGRESS, IN_REVIEW, BLOCKED, DONE and ARCHIVED.

BR-08-R02 BACKLOG may become READY; READY may become IN_PROGRESS; IN_PROGRESS may become IN_REVIEW or BLOCKED.

BR-08-R03 IN_REVIEW may become DONE or return to IN_PROGRESS; BLOCKED may return to IN_PROGRESS.

BR-08-R04 Only a Project Manager can reopen DONE to IN_PROGRESS, with a mandatory reason.

BR-08-R05 A task may be ARCHIVED only when it has been DONE for at least 30 days and is not linked as an unresolved dependency.

BR-08-R06 Forbidden transitions return INVALID_TRANSITION and leave the record unchanged.

Worked business scenario

A contributor submits code for review and moves a task from IN_PROGRESS to IN_REVIEW; a reviewer requests changes by returning it to IN_PROGRESS.

Boundary and rejection example

An attempted BACKLOG-to-DONE shortcut is rejected even when the caller is a workspace OWNER.

Test and implementation interpretation

Use BR-08-R01 through BR-08-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-09 · SOURCE PAGE 009/100

Task assignment and ownership

Policy intent: Ownership is explicit and auditable.

Normative business rules

BR-09-R01 Each task has zero or one primary assignee and always has one reporter.

BR-09-R02 An assignee must have CONTRIBUTOR or Project Manager access to that project.

BR-09-R03 Project Managers may assign any eligible project member; contributors may claim unassigned tasks.

BR-09-R04 Changing assignee records old and new user IDs and sends an in-app event to the new assignee.

BR-09-R05 An assignee removed from the project is cleared from open tasks, which remain in their current states.

BR-09-R06 A completed task keeps its historical assignee in the audit trail when that user leaves the project.

Worked business scenario

MOB-0084 is unassigned until a contributor claims it. The previous reporter remains the reporter after the claim.

Boundary and rejection example

Trying to assign a task to a workspace member who has no project access returns ASSIGNEE_NOT_ELIGIBLE.

Test and implementation interpretation

Use BR-09-R01 through BR-09-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Priority and severity

Policy intent: Priorities describe business urgency, not workflow status.

Normative business rules

BR-10-R01 Allowed priority codes are P0-CRITICAL, P1-HIGH, P2-NORMAL and P3-LOW.

BR-10-R02 New tasks default to P2-NORMAL unless a task template provides a specific default.

BR-10-R03 Only a Project Manager can set P0-CRITICAL; contributors may propose a change as a comment.

BR-10-R04 P0 tasks require an assigned Project Manager and an impact summary of at least 20 characters.

BR-10-R05 Priority changes are audited but do not automatically change state, due date or sprint.

BR-10-R06 Board sorting uses priority and then creation time when the user explicitly chooses priority ordering.

Worked business scenario

A login outage is logged as a P0 bug with business impact documented and a manager responsible for triage.

Boundary and rejection example

P0 without an impact summary is rejected rather than silently downgraded to P1.

Test and implementation interpretation

Use BR-10-R01 through BR-10-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Dependencies between tasks

Policy intent: Dependencies model explicit blockers and must remain acyclic.

Normative business rules

BR-11-R01 Dependency types are BLOCKS and RELATES_TO; BLOCKS is directional while RELATES_TO is symmetric.

BR-11-R02 BLOCKS may link tasks only within the same workspace; project boundaries do not remove authorization checks.

BR-11-R03 Self-dependencies and cycles in the BLOCKS graph are rejected.

BR-11-R04 A task cannot transition to DONE while any task blocking it is not DONE.

BR-11-R05 Deleting a task does not silently remove historical dependency events; task hard deletion is not available.

BR-11-R06 Users may read dependency labels only for linked tasks they are authorized to view.

Worked business scenario

When ALPHA-2 blocks ALPHA-9, ALPHA-9 cannot be completed until ALPHA-2 reaches DONE.

Boundary and rejection example

Creating A BLOCKS B after B already BLOCKS A returns DEPENDENCY_CYCLE.

Test and implementation interpretation

Use BR-11-R01 through BR-11-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Due dates and estimates

Policy intent: Planning fields are independent from state transitions.

Normative business rules

BR-12-R01 A due date is optional and represented as a local calendar date plus the project time zone for display.

BR-12-R02 Due dates may not precede the task creation date without a Project Manager override reason.

BR-12-R03 Estimates are stored as positive integer minutes and may be changed until a task becomes DONE.

BR-12-R04 Contributors may update their assigned task estimate; managers may update any task estimate.

BR-12-R05 An estimate adjustment does not change previously logged actual time.

BR-12-R06 Overdue status is derived from the project time zone and must not be stored as a separately editable task state.

Worked business scenario

A task due on 2026-11-12 is overdue after that calendar date in the project time zone if it is not DONE.

Boundary and rejection example

A negative estimate or malformed date fails validation and leaves the saved plan unchanged.

Test and implementation interpretation

Use BR-12-R01 through BR-12-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-13 · SOURCE PAGE 013/100

Subtasks and task grouping

Policy intent: Subtasks allow limited decomposition without creating a second project hierarchy.

Normative business rules

BR-13-R01 TASK and STORY items may have subtasks; EPIC items contain top-level tasks through epic links rather than parent-child subtasks.

BR-13-R02 Subtasks have their own assignee, state, estimate and audit history.

BR-13-R03 Subtask nesting is limited to one level below a parent task.

BR-13-R04 A parent task cannot move to DONE while any nonarchived subtask is not DONE.

BR-13-R05 Removing a subtask association does not delete the subtask; it becomes a top-level task.

BR-13-R06 Parent and subtask must belong to the same project.

Worked business scenario

A parent feature task contains separate UI and API subtasks, both of which must be DONE before parent completion.

Boundary and rejection example

Trying to set a grandchild task parent or link across projects returns INVALID_PARENT.

Test and implementation interpretation

Use BR-13-R01 through BR-13-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Labels and custom fields

Policy intent: Project metadata improves filtering without changing task authorization.

Normative business rules

BR-14-R01 Labels are lowercase slug values of 2-30 characters and are unique by value within one project.

BR-14-R02 Each task may have at most 20 labels; removing a label from a project removes task associations after confirmation.

BR-14-R03 Custom fields are defined per project and support TEXT, NUMBER, BOOLEAN and SINGLE_SELECT types.

BR-14-R04 Only Project Managers may create or change custom field definitions.

BR-14-R05 Required custom fields must be populated before a task enters READY.

BR-14-R06 Changing a field type after any task has a value is prohibited; create a replacement field instead.

Worked business scenario

A manager adds the single-select field Component with options mobile, backend and infra; board filters can use it.

Boundary and rejection example

Importing an unknown option for a SINGLE_SELECT field returns INVALID_FIELD_OPTION.

Test and implementation interpretation

Use BR-14-R01 through BR-14-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-15 · SOURCE PAGE 015/100

Comments, edits and mentions

Policy intent: Conversations are attributed and preserved as project records.

Normative business rules

BR-15-R01 Contributors and Project Managers may comment on visible, nonarchived tasks.

BR-15-R02 A comment supports Markdown of at most 10,000 characters, with script and unsafe HTML stripped during rendering.

BR-15-R03 Authors may edit their own comments within 30 minutes of creation; managers may redact for policy violations.

BR-15-R04 Each edit records a version and timestamp; a redacted comment retains an audit marker.

BR-15-R05 Mentions notify only active users who currently have permission to view the task.

BR-15-R06 Comment deletion is logical; historical author and event IDs remain in the audit trail.

Worked business scenario

A developer posts a review note and mentions the assignee. The mentioned assignee receives an in-app alert only if they have access.

Boundary and rejection example

An attempted edit outside the author's edit window fails with COMMENT_EDIT_WINDOW_EXPIRED.

Test and implementation interpretation

Use BR-15-R01 through BR-15-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-16 · SOURCE PAGE 016/100

Attachments and downloads

Policy intent: Files inherit the security context of their parent task.

Normative business rules

BR-16-R01 A task attachment is at most 20 MiB and requires an allowed MIME type from the workspace policy.

BR-16-R02 Attachment filenames are displayed as sanitized text and never evaluated as markup.

BR-16-R03 Contributors may upload to projects where they can edit tasks; observers may download but cannot upload.

BR-16-R04 Download URLs are short-lived and scoped to the current authorized principal.

BR-16-R05 Deleting an attachment removes the active file reference and records an audit event.

BR-16-R06 Antivirus scanning must complete successfully before an uploaded file becomes downloadable.

Worked business scenario

A user uploads a 5 MiB screenshot to a bug; the UI shows a scanning state until the file is safe to open.

Boundary and rejection example

An upload exceeding 20 MiB returns FILE_TOO_LARGE without attaching partial content to the task.

Test and implementation interpretation

Use BR-16-R01 through BR-16-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-17 · SOURCE PAGE 017/100

Sprint creation and capacity

Policy intent: Sprints group planned tasks for a fixed delivery interval.

Normative business rules

BR-17-R01 Only a Project Manager may create a sprint or edit its dates.

BR-17-R02 A sprint belongs to one project and has an inclusive start date and exclusive end date.

BR-17-R03 Sprint duration must be from 7 to 28 days inclusive.

BR-17-R04 Sprints begin in PLANNED state and can be scheduled back-to-back without overlap.

BR-17-R05 Capacity is expressed in estimated minutes for assigned contributors; capacity is advisory, not an automatic task block.

BR-17-R06 A task can belong to at most one not-yet-completed sprint at a time.

Worked business scenario

A Project Manager creates a two-week sprint with 3,000 minutes of planned work and reviews capacity warnings.

Boundary and rejection example

A six-day sprint or one overlapping an existing PLANNED sprint is rejected with INVALID_SPRINT_DATES.

Test and implementation interpretation

Use BR-17-R01 through BR-17-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Sprint execution and completion

Policy intent: Sprint state is enforced at the project level.

Normative business rules

BR-18-R01 Sprint states are PLANNED, ACTIVE and COMPLETED; COMPLETED is terminal.

BR-18-R02 Only one sprint may be ACTIVE per project at a time.

BR-18-R03 Activating a sprint records a snapshot of its committed task IDs and initial estimates.

BR-18-R04 When a sprint is completed, unfinished tasks are returned to the project backlog, retaining task state and history.

BR-18-R05 Completed sprint reports use commitment snapshot values rather than silently rewriting historical estimates.

BR-18-R06 Only a Project Manager may complete a sprint; this action cannot be rolled back by the API.

Worked business scenario

A manager closes a sprint that contains three DONE and two IN_PROGRESS tasks; the two open tasks return to backlog.

Boundary and rejection example

Starting a second sprint while one is ACTIVE returns ACTIVE_SPRINT_EXISTS.

Test and implementation interpretation

Use BR-18-R01 through BR-18-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Backlog and ordering

Policy intent: Backlog order is an explicit per-project sequence.

Normative business rules

BR-19-R01 Unscheduled tasks appear in the backlog when their state is not ARCHIVED.

BR-19-R02 The Project Manager may reorder backlog items and save the resulting sequence.

BR-19-R03 Ordering tokens are generated by the server and are not interpreted as business priority.

BR-19-R04 Contributor drag-and-drop ordering is visible locally but requires manager permission to persist.

BR-19-R05 Adding a task to an active sprint is allowed only to a Project Manager and creates a scope-change audit event.

BR-19-R06 Removing a DONE task from a completed sprint is unsupported.

Worked business scenario

A manager moves a high-priority bug above a feature in backlog without altering either task's P-code.

Boundary and rejection example

Concurrent reorder requests are checked against an ordering version to avoid silently overwriting changes.

Test and implementation interpretation

Use BR-19-R01 through BR-19-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Boards and saved views

Policy intent: Boards visualize authorized task data rather than store separate tasks.

Normative business rules

BR-20-R01 Each project has a canonical board with columns derived from allowed task states.

BR-20-R02 Board counts include only tasks visible under the caller's current project permissions.

BR-20-R03 Saved filters may reference state, assignee, priority, labels and sprint; no arbitrary code filters are accepted.

BR-20-R04 Users may save private views, while managers may publish project-wide views.

BR-20-R05 State changes initiated from a board use the same transition validation as the task API.

BR-20-R06 Archived tasks are hidden by default but can be included by an authorized explicit filter.

Worked business scenario

A contributor saves My Active Work filtered by their user ID and states IN_PROGRESS and BLOCKED.

Boundary and rejection example

Dragging BACKLOG directly into DONE cannot bypass the state machine and displays a validation error.

Test and implementation interpretation

Use BR-20-R01 through BR-20-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Notifications and delivery

Policy intent: Events are generated from committed domain changes.

Normative business rules

BR-21-R01 Notification channels are IN_APP and EMAIL; direct push notifications are outside baseline scope.

BR-21-R02 Task assignment, mentions, sprint activation and due-date alerts may generate notifications.

BR-21-R03 In-app notifications are enabled for every eligible user and can be marked as read.

BR-21-R04 Mention email notifications are disabled by default until a user explicitly enables them.

BR-21-R05 Notification payloads never reveal content from a project the recipient cannot currently access.

BR-21-R06 Notification worker retries must not duplicate in-app notification records for the same event and recipient.

Worked business scenario

An assignee receives one in-app event when a task is assigned; email is sent only when the relevant preference is enabled.

Boundary and rejection example

If a user loses access before an email is sent, protected task details are omitted from the outgoing message.

Test and implementation interpretation

Use BR-21-R01 through BR-21-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Activity history and audit

Policy intent: Critical updates generate immutable activity events.

Normative business rules

BR-22-R01 Task creation, state changes, reassignment, comment edit, attachment removal and permission changes are audited.

BR-22-R02 Audit event fields include event ID, actor ID, workspace ID, subject ID, type and UTC timestamp.

BR-22-R03 Audit payloads record necessary changed fields but never passwords, tokens or full file contents.

BR-22-R04 Ordinary contributors can read task-level activity for tasks they can read; workspace audits require ADMIN or OWNER.

BR-22-R05 Events are append-only and cannot be rewritten through public APIs.

BR-22-R06 Audited events preserve references to deactivated accounts without restoring those accounts' access.

Worked business scenario

A manager reviews who reopened a task, sees the previous state and supplied reason, and traces the request correlation ID.

Boundary and rejection example

The system must not accept a client-supplied actor ID as authoritative audit identity.

Test and implementation interpretation

Use BR-22-R01 through BR-22-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Search and query semantics

Policy intent: Search is permission-aware and favors predictable behavior.

Normative business rules

BR-23-R01 Search indexes task keys, titles, descriptions and comment text only after those records are committed.

BR-23-R02 Search always scopes by workspace and filters results by project membership before display.

BR-23-R03 Full-text matching is case-insensitive; quoted phrases require contiguous normalized words.

BR-23-R04 Facets supported in baseline are project, task type, priority, state and assignee.

BR-23-R05 Search results sort by relevance unless the caller explicitly requests a supported chronological sort.

BR-23-R06 Deleted or archived records follow the caller's explicit include_archived filter.

Worked business scenario

Searching for broken login retrieves authorized tickets with the phrase in title or description, but never leaks task titles from a private project.

Boundary and rejection example

Unknown filter field names return INVALID_FILTER rather than executing arbitrary expressions.

Test and implementation interpretation

Use BR-23-R01 through BR-23-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-24 · SOURCE PAGE 024/100

Reports and computed metrics

Policy intent: Reports derive from task and sprint events, not editable counters.

Normative business rules

BR-24-R01 Available reports are sprint summary, cycle-time overview, workload per assignee and project completion trend.

BR-24-R02 Cycle time is measured from the first IN_PROGRESS timestamp to the first DONE timestamp in UTC.

BR-24-R03 Reopened tasks show a reopen count separately; prior completion events are never erased.

BR-24-R04 Project reports include only authorized project data and suppress user names from inaccessible projects.

BR-24-R05 Aggregated report data refreshes at least once every 15 minutes during normal service.

BR-24-R06 Every report includes an as_of timestamp so viewers can assess data freshness.

Worked business scenario

A manager opens a sprint summary and sees completed versus committed story counts, including a report freshness timestamp.

Boundary and rejection example

A report request for a project outside the caller’s membership returns NOT_FOUND without aggregate leakage.

Test and implementation interpretation

Use BR-24-R01 through BR-24-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Time tracking

Policy intent: Actual effort is captured as explicit user entries.

Normative business rules

BR-25-R01 A worklog entry has an owning task, author, start date, minutes spent and optional note.

BR-25-R02 Minutes spent must be from 1 to 1,440; worklogs may not represent future local dates.

BR-25-R03 Contributors may add worklogs to tasks they are assigned or manage; Project Managers may log on project tasks.

BR-25-R04 Only the author may edit a worklog within 24 hours; managers may correct later entries with a reason.

BR-25-R05 Worklogs remain after task completion and contribute to actual-effort reports.

BR-25-R06 Deleted worklogs are soft-deleted and removed from totals while remaining in audit history.

Worked business scenario

A contributor logs 90 minutes spent debugging and the task's actual effort becomes the sum of active logs.

Boundary and rejection example

The API rejects a zero-minute log and never interprets a missing duration as eight hours.

Test and implementation interpretation

Use BR-25-R01 through BR-25-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Milestones and releases

Policy intent: Milestones group delivery objectives across tasks.

Normative business rules

BR-26-R01 Project Managers may define milestones with unique project-local names and target dates.

BR-26-R02 Tasks may be linked to multiple milestones, and links do not change their workflow states.

BR-26-R03 A milestone is COMPLETE only when every nonarchived linked task is DONE.

BR-26-R04 Release records have a version label, planned date and state PLANNED, PUBLISHED or CANCELLED.

BR-26-R05 Publishing a release stores an immutable snapshot of included task keys and milestone status.

BR-26-R06 Cancelling a release does not reopen tasks that were already DONE.

Worked business scenario

A release 2.4 contains a fixed list of task keys at publication and remains auditable even if their titles change later.

Boundary and rejection example

A milestone cannot be marked complete while a linked task remains IN_REVIEW.

Test and implementation interpretation

Use BR-26-R01 through BR-26-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

External guests and sharing

Policy intent: Guest access is narrow and never public-link based.

Normative business rules

BR-27-R01 Only workspace OWNERs and ADMINs may invite a GUEST into selected projects.

BR-27-R02 Guests may view assigned project tasks, read comments and download approved attachments.

BR-27-R03 Guests cannot create, comment, edit, export projects or invite other users.

BR-27-R04 Guests never see workspace-level membership directories or private project existence.

BR-27-R05 A GUEST invitation must identify at least one project before it can be accepted.

BR-27-R06 Guest access expires at the membership expiry date if one was configured by the issuer.

Worked business scenario

A customer stakeholder is invited as a guest to review a project board but cannot move tasks or export reports.

Boundary and rejection example

A guest opening a direct link to an unrelated workspace project sees NOT_FOUND.

Test and implementation interpretation

Use BR-27-R01 through BR-27-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-28 · SOURCE PAGE 028/100

Templates and automated actions

Policy intent: Task templates standardize content without bypassing approvals.

Normative business rules

BR-28-R01 Project Managers may create a project-scoped template specifying task type, title pattern and acceptance-criteria checklist.

BR-28-R02 Instantiating a template always produces a new task with a new immutable task key.

BR-28-R03 Template-instantiated tasks use P2-NORMAL as their default priority unless the caller overrides it with an authorized value.

BR-28-R04 Template edits apply only to subsequently created tasks, never retroactively to existing tasks.

BR-28-R05 Template automation may prefill fields but may not automatically mark a task DONE.

BR-28-R06 If a required project custom field is missing, template instantiation returns FIELD_REQUIRED rather than creating an invalid READY task.

Worked business scenario

A Project Manager uses a regression-bug template to create a new BACKLOG task with a structured reproduction checklist.

Boundary and rejection example

A template cannot grant a role, import another project's private content or skip required validations.

Test and implementation interpretation

Use BR-28-R01 through BR-28-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Archiving and exports

Policy intent: Archiving protects history while limiting active changes.

Normative business rules

BR-29-R01 Project Managers may request a project export in JSON or CSV format; workspace admins may export any project.

BR-29-R02 An ARCHIVED project remains readable to authorized members but rejects new comments, tasks and sprint changes.

BR-29-R03 An archived project can be restored to ON_HOLD within 14 days by a workspace ADMIN or OWNER.

BR-29-R04 After the restore window, the archive remains available for read and export but cannot be reactivated through the UI.

BR-29-R05 Exports include selected tasks, comments, permitted user references and stable document IDs.

BR-29-R06 Export jobs write an audit event, expire download links after 24 hours and must not expose another tenant’s objects.

Worked business scenario

A project manager exports their own completed project to CSV while preparing a handover package.

Boundary and rejection example

Requesting restore on the fifteenth day returns RESTORE_WINDOW_EXPIRED.

Test and implementation interpretation

Use BR-29-R01 through BR-29-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Webhooks and integrations

Policy intent: Outbound webhooks notify external systems of committed events.

Normative business rules

BR-30-R01 Only workspace ADMIN or OWNER may configure a webhook endpoint.

BR-30-R02 Webhook subscriptions may include task.created, task.updated, sprint.started and sprint.completed events.

BR-30-R03 Each webhook request includes a stable event ID and an HMAC signature based on the configured secret.

BR-30-R04 Webhook delivery uses an asynchronous queue and never blocks the source task mutation.

BR-30-R05 Failed delivery attempts record status code and next-attempt timestamp for inspection.

BR-30-R06 Webhook payloads include workspace-scoped identifiers but not access tokens or private attachment download URLs.

Worked business scenario

An integration receives task.updated twice after an HTTP timeout; it can deduplicate by event ID.

Boundary and rejection example

Invalid endpoint URLs or loopback destinations are rejected to reduce server-side request forgery.

Test and implementation interpretation

Use BR-30-R01 through BR-30-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Retention and account deletion

Policy intent: Retention balances collaboration and privacy.

Normative business rules

BR-31-R01 Workspace audit events are retained for 365 days after creation unless legal retention requirements supersede.

BR-31-R02 Soft-deleted comments and worklogs are excluded from active views but retain audit references.

BR-31-R03 An account deactivation immediately invalidates active sessions and prevents new workspace actions.

BR-31-R04 Project content authored by a deactivated user is preserved with a deactivated-author marker.

BR-31-R05 Workspace OWNERs may request a workspace export before permanent deletion of the workspace.

BR-31-R06 Completed deletion jobs publish a verifiable job status without returning deleted private data.

Worked business scenario

A departing engineer is deactivated; their historical task comments remain attributable without granting login access.

Boundary and rejection example

An unauthenticated caller cannot use a deletion job ID to discover a workspace’s contents.

Test and implementation interpretation

Use BR-31-R01 through BR-31-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-32 · SOURCE PAGE 032/100

Security and trust boundaries

Policy intent: Security invariants apply equally to UI, APIs and background jobs.

Normative business rules

BR-32-R01 All workspace reads, writes and exports perform tenant scoping from trusted server context.

BR-32-R02 Client-reported roles, actor IDs and workspace IDs are never trusted without authorization checks.

BR-32-R03 Security-sensitive actions require authentication and are logged with a correlation ID.

BR-32-R04 Secrets are stored in a secret-management system, never in project settings exports.

BR-32-R05 Attachment scanning and Markdown sanitization are mandatory defenses for user-provided content.

BR-32-R06 Rate limits apply to login, invitations and bulk exports and return explicit retry guidance when enforced.

Worked business scenario

A crafted API request changes workspace_id but cannot read or mutate a task owned by a different tenant.

Boundary and rejection example

Sensitive information is omitted from logs and public error messages even when a request fails.

Test and implementation interpretation

Use BR-32-R01 through BR-32-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

01 / BUSINESS RULES / PRODUCT REQUIREMENTS · BR-33 · SOURCE PAGE 033/100

Imports and external migrations

Policy intent: Structured imports are validated before they create objects.

Normative business rules

BR-33-R01 Workspace ADMINs may import projects from a versioned CSV or JSON bundle.

BR-33-R02 Import previews identify invalid task keys, unknown assignees, invalid statuses and broken dependencies.

BR-33-R03 An import does not publish partial data if mandatory validation fails; the caller receives a validation report.

BR-33-R04 Original external identifiers are kept in an optional import_reference field for traceability.

BR-33-R05 Imported task states must map to an allowed TeamFlow state; unmapped states are not silently coerced.

BR-33-R06 Every accepted import produces an audit record with source format, counts and actor.

Worked business scenario

An admin previews a 100-task CSV, fixes three unknown user references and then commits a valid batch.

Boundary and rejection example

A cyclic dependency in the import aborts the commit and reports the offending external IDs.

Test and implementation interpretation

Use BR-33-R01 through BR-33-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Accessibility and localization

Policy intent: The interface supports keyboard and translated presentation without changing domain values.

Normative business rules

BR-34-R01 Critical task operations are operable using a keyboard without drag-and-drop.

BR-34-R02 UI controls have accessible names, visible focus indication and programmatic error descriptions.

BR-34-R03 Domain status and priority codes remain invariant English identifiers in APIs.

BR-34-R04 Date display follows user locale and project time zone while server event timestamps remain UTC.

BR-34-R05 User-entered task titles and comments are stored as written and are not automatically translated.

BR-34-R06 Localization of UI strings must not change API error codes or audit event identifiers.

Worked business scenario

A keyboard-only user can move a task to IN_REVIEW through an accessible action menu rather than dragging it across a board.

Boundary and rejection example

A Romanian UI can display a translated label for IN_PROGRESS while APIs continue to return IN_PROGRESS.

Test and implementation interpretation

Use BR-34-R01 through BR-34-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Plans, quotas and usage

Policy intent: Workspace subscription tier changes capacity rather than authorization rules.

Normative business rules

BR-35-R01 FREE workspaces can have up to three ACTIVE projects and 15 active members.

BR-35-R02 TEAM workspaces can have up to 25 ACTIVE projects and 100 active members.

BR-35-R03 ENTERPRISE quotas are configured by contract and reported by the usage API.

BR-35-R04 Quota checks occur when activating projects or inviting members, not when reading historical records.

BR-35-R05 Downgrading below current usage blocks new capacity-consuming actions but does not delete existing data.

BR-35-R06 Plan status is visible only to OWNER and ADMIN; ordinary project members cannot change billing settings.

Worked business scenario

A TEAM workspace downgraded to FREE with five active projects can read all five but cannot activate a sixth.

Boundary and rejection example

A quota violation returns PLAN_LIMIT_REACHED with the relevant resource type, without silently altering the plan.

Test and implementation interpretation

Use BR-35-R01 through BR-35-R06 as individual, citable policy statements. A caller must not infer permission from UI visibility; the applicable authorization and validation rules are checked before committing state changes. Concrete examples illustrate rather than replace the normative rules above.

Cross-cutting source: BR-01 (tenant scope), BR-04 (role authorization), BR-22 (audit history) and GL-02 (stable error codes), where applicable.

Create and verify an account

Primary actor: Visitor. Preconditions: A nonregistered visitor has an email address and can receive verification mail.

Main success flow

01 Open registration and enter email, password and accepted terms.

02 Server normalizes email, checks uniqueness and creates an unverified account.

03 Server sends a single-use verification link with expiration.

04 Visitor follows the link; server validates token and verifies email.

05 Visitor signs in; the UI shows an empty workspace switcher until an invitation is accepted.

06 Client does not request project data before membership is established.

Alternative flow and error handling

Duplicate email returns EMAIL_ALREADY_REGISTERED; an expired link returns VERIFICATION_TOKEN_EXPIRED; a replayed token cannot verify a second account.

Observable acceptance result

The verified account has no implicit workspace role or project access.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-02; BR-03; API-01.

Sign in, refresh and revoke sessions

Primary actor: Registered user. Preconditions: The user account is verified and not locked.

Main success flow

01 User submits normalized email and password over a secure connection.

02 Server checks credentials and lockout state without leaking account existence.

03 Server creates one session and returns access and rotating refresh credentials.

04 Client uses access token for workspace requests and refreshes only when required.

05 On logout, server revokes the current refresh token and deletes associated active session.

06 Using revoked or reused tokens requires a new sign-in.

Alternative flow and error handling

Five failed passwords invoke a fifteen-minute lockout; an expired session returns SESSION_EXPIRED; refresh-token reuse revokes its associated session.

Observable acceptance result

Access fails after revocation and audit contains a session-revoked event without credential contents.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-03; BR-32; API-01.

Invite a workspace member

Primary actor: Workspace Admin. Preconditions: The target workspace exists and the admin has invitation rights.

Main success flow

01 Admin opens member management and selects Invite user.

02 Admin enters email, desired workspace role and optional invited projects.

03 Server checks role permissions, current plan quotas and existing memberships.

04 Invitation record is created with a one-use token and seven-day expiry.

05 Recipient receives the link and accepts while signed into the matching email account.

06 Workspace membership and project grants appear after one successful acceptance.

Alternative flow and error handling

An invite for the wrong verified email is denied; role OWNER requires owner authorization; expired or revoked links cannot be reused.

Observable acceptance result

Invitation creates at most one membership and records who invited whom.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-02; BR-04; BR-35; API-02.

Create and activate a project

Primary actor: Project Manager / Workspace Admin. Preconditions: Caller has permission to create projects, and workspace plan has free active-project capacity.

Main success flow

01 User opens New project and supplies name, unique key, description and project time zone.

02 Server validates syntax and key uniqueness in the selected workspace.

03 Project is created in DRAFT and creator becomes a Project Manager.

04 Manager configures project visibility and invites at least one project contributor.

05 Manager activates project; server validates manager existence and quota.

06 Board and backlog endpoints begin returning the new active project.

Alternative flow and error handling

Invalid or duplicate key returns an explanatory error; activation without a manager or above plan quota fails atomically.

Observable acceptance result

Project key is stable and activity records creation and activation.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-05; BR-06; BR-35; API-03.

Manage project membership

Primary actor: Project Manager. Preconditions: Caller is a Project Manager or a permitted workspace administrator.

Main success flow

01 Manager opens project Members and sees current project roles.

02 Manager selects an existing workspace member and project role.

03 Server checks the invitee’s active workspace membership and project capacity.

04 Project membership is saved and visible to authorized collaborators.

05 Manager can later change or remove project membership.

06 Existing task activity remains historically attributable if a user is removed.

Alternative flow and error handling

A user without workspace membership cannot be added directly; removal immediately revokes access and clears assignment on open tasks.

Observable acceptance result

Project member list and access checks reflect role changes in the next request.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-06; BR-09; API-04.

Create a task from scratch or template

Primary actor: Contributor / Project Manager. Preconditions: Project is ACTIVE and user has edit permission.

Main success flow

01 User chooses task type, title and optional template.

02 Client renders type-specific fields and applicable custom field definitions.

03 User adds description, assignee, estimate and relevant acceptance criteria.

04 Server validates title, project membership and required fields.

05 Server allocates the next permanent PROJECTKEY-NNNN identifier and initial BACKLOG state.

06 Client opens task detail with creation audit entry and project context.

Alternative flow and error handling

STORY missing criteria may be created in BACKLOG but cannot move to READY; template defaults currently display P3-LOW in the form specification.

Observable acceptance result

Exactly one new task is created, with no unauthorized fields or cross-project associations.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-07; BR-14; BR-28; API-06.

Advance task through review

Primary actor: Assignee / Project Manager. Preconditions: Task exists in an editable project and caller can perform the chosen transition.

Main success flow

01 Assignee opens a READY task and selects Start work.

02 Server checks transition READY-to-IN_PROGRESS and records timestamp.

03 Assignee selects Request review, causing IN_PROGRESS-to-IN_REVIEW.

04 Reviewer may send back to IN_PROGRESS or manager may approve to DONE.

05 Server checks every blocking dependency and unfinished subtask before DONE.

06 Board and activity history update after successful commit.

Alternative flow and error handling

Direct BACKLOG-to-DONE fails; unresolved blockers fail completion; reopen of DONE requires Project Manager and reason.

Observable acceptance result

UI cannot bypass the server state machine even via drag-and-drop.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-08; BR-11; BR-13; API-07.

Assign or claim a task

Primary actor: Project Manager / Contributor. Preconditions: Project membership is active and task is not archived.

Main success flow

01 Manager opens assignee control and searches eligible project contributors.

02 Client displays only users eligible to be primary assignee.

03 Manager selects a user and submits the change.

04 Server validates membership and writes the old and new assignee values.

05 One in-app assignment notification is enqueued for the new assignee.

06 Open task and project activity streams show the updated responsibility.

Alternative flow and error handling

Contributors may self-assign only if task was unassigned; selecting observer or guest returns ASSIGNEE_NOT_ELIGIBLE.

Observable acceptance result

Assignment changes are traceable to an authenticated actor and do not change task state.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-09; BR-21; API-08.

Create and resolve a blocking link

Primary actor: Contributor / Project Manager. Preconditions: Both referenced tasks exist in the workspace and are visible to the caller.

Main success flow

01 User opens Dependencies and chooses another task as blocker.

02 Server verifies access to both tasks and checks for self-links.

03 Server checks if the proposed edge creates a directed cycle.

04 Server saves BLOCKS and records an activity entry.

05 Completion of the dependent task is prevented while blocker is not DONE.

06 After blocker reaches DONE, dependent task may pass other completion checks.

Alternative flow and error handling

Cross-workspace links fail; cyclic links return DEPENDENCY_CYCLE; unauthorized target keys are not exposed through error text.

Observable acceptance result

Acyclic links control completion but do not silently change workflow states.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-11; API-09.

Plan an upcoming sprint

Primary actor: Project Manager. Preconditions: Project is ACTIVE and no planned interval overlaps the proposed interval.

Main success flow

01 Manager opens sprint planning and sets a date range within 7-28 days.

02 Server validates inclusive start and exclusive end under project time zone.

03 Manager assigns eligible backlog tasks to the PLANNED sprint.

04 UI calculates estimates by assignee and highlights advisory capacity overages.

05 Manager saves the sprint and its task associations.

06 Backlog indicates each task's planned sprint without altering task state.

Alternative flow and error handling

An overlapping or overlong interval fails; exceeding capacity shows warning but does not prevent saving.

Observable acceptance result

Every task is assigned to no more than one incomplete sprint.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-17; BR-19; API-10.

Activate a sprint and freeze commitment

Primary actor: Project Manager. Preconditions: A PLANNED sprint exists and no sprint is ACTIVE in the same project.

Main success flow

01 Manager chooses Start sprint on the sprint overview.

02 Server checks project state and active-sprint uniqueness.

03 Server copies current task IDs and estimates into a commitment snapshot.

04 Sprint state becomes ACTIVE in a single committed operation.

05 Audit and notification events are enqueued asynchronously.

06 Sprint dashboard displays commitment baseline and current progress separately.

Alternative flow and error handling

Trying to activate while another sprint is active returns ACTIVE_SPRINT_EXISTS; failed snapshot write rolls back activation.

Observable acceptance result

Sprint commitment remains immutable after activation, even if tasks are added as scope changes.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-17; BR-18; API-10.

Complete a sprint with unfinished tasks

Primary actor: Project Manager. Preconditions: Sprint is ACTIVE and project manager initiated close.

Main success flow

01 Manager opens Sprint completion and reviews DONE and unfinished items.

02 Server computes completion metrics from commitment snapshot and task events.

03 Server marks the sprint COMPLETED and locks its historical membership.

04 Unfinished tasks are automatically assigned to the next PLANNED sprint if available.

05 Completed tasks remain linked to completed sprint reports.

06 UI displays completion report and backlog/sprint redistribution.

Alternative flow and error handling

A second completion request does not repeat mutations; a nonmanager cannot close the sprint.

Observable acceptance result

Historical completion is stable; downstream task locations reflect the specified rollover behavior.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-18; BR-19; API-11.

Post a comment and mention colleagues

Primary actor: Contributor / Project Manager. Preconditions: Task is visible and editable; mentioned people may have different project grants.

Main success flow

01 Author writes Markdown comment and types @ to select a colleague.

02 Client suggests only users visible within the project membership directory.

03 Server sanitizes content and stores authored comment with UTC timestamp.

04 Server resolves mention recipients against current task visibility.

05 An in-app event is enqueued per eligible recipient.

06 Editing own comment within allowed window creates a new comment version.

Alternative flow and error handling

Mentioning a user without task access creates no notification; edit outside author window returns COMMENT_EDIT_WINDOW_EXPIRED.

Observable acceptance result

Every edit remains auditable and no unsafe script executes when the comment is displayed.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-15; BR-21; API-12.

Upload and inspect an attachment

Primary actor: Contributor / Observer. Preconditions: A task exists in an ACTIVE project and caller has the required read/write role.

Main success flow

01 Contributor selects a file from a task attachment area.

02 Client requests an authorized upload slot with filename and size.

03 Server validates declared MIME, file size and task write permissions.

04 File is uploaded into a quarantine object store and antivirus scanning begins.

05 After successful scan, attachment becomes available through a short-lived download URL.

06 Observer may download a clean attachment but cannot upload.

Alternative flow and error handling

Blocked MIME or oversize upload is rejected; failed scan marks attachment REJECTED and disallows downloads.

Observable acceptance result

Private objects remain inaccessible without renewed server-side permission checks.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-16; BR-32; API-12.

Filter and personalize board view

Primary actor: Project member. Preconditions: Caller can read the target project board.

Main success flow

01 User opens board and chooses state, label, assignee or priority filters.

02 Client sends an explicit structured filter set with project scope.

03 Server validates allowed filter keys and authorization.

04 Board columns show only matching, viewable tasks and accurate counts.

05 User optionally saves filter privately or manager publishes it project-wide.

06 Reopening the view retrieves saved criteria without changing canonical task data.

Alternative flow and error handling

Unknown filter keys fail with INVALID_FILTER; saved views referencing deleted labels drop the invalid criterion with a visible warning.

Observable acceptance result

Board drag actions must still call the normal task transition endpoint.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-20; BR-23; API-14.

Search workspace tasks safely

Primary actor: Workspace member. Preconditions: User has permission for at least one project in the selected workspace.

Main success flow

01 User enters a phrase or task key into the search interface.

02 Client submits workspace-scoped query and optional supported facets.

03 Server searches indexed title, description and comment text.

04 Server applies current project permissions to every matched record.

05 Server returns scored results with task key, title, permitted snippet and paging cursor.

06 User opens a result only after another resource authorization check.

Alternative flow and error handling

Unknown facet returns INVALID_FILTER; index delay may briefly omit a newly created task but never reveal unauthorized text.

Observable acceptance result

No cross-tenant or private-project metadata appears in search counts, snippets or suggestions.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-01; BR-23; API-14.

Configure notification preferences

Primary actor: Registered workspace member. Preconditions: Member has a valid active account and notification settings access.

Main success flow

01 Member opens account preferences and selects email versus in-app events.

02 Client shows separate settings for assignment, mention, due-date and sprint events.

03 Member updates a preference and sends the new configuration.

04 Server validates event/channel name and stores an account-scoped preference.

05 New domain events use the updated preference after commit.

06 Member can mark an in-app notification read without affecting its audit source event.

Alternative flow and error handling

An invalid event key is rejected; revoking project access suppresses content regardless of saved preferences.

Observable acceptance result

In-app event identity prevents duplicated records under delivery retry.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-21; API-13.

Log actual time against a task

Primary actor: Contributor / Project Manager. Preconditions: Caller may edit the task or is an authorized manager.

Main success flow

01 User opens Time tracking and enters local work date, duration and optional note.

02 Server converts date under project time zone and validates minutes from 1 to 1440.

03 User saves a new worklog with actor and ownership references.

04 Task actual effort is recomputed as the sum of active worklogs.

05 Author may edit recent worklog during the permitted edit window.

06 Task report shows estimated and actual time as distinct quantities.

Alternative flow and error handling

Future work date, negative duration and unauthorized edits fail with explicit error codes; no automatic eight-hour default exists.

Observable acceptance result

Actual effort remains traceable to entries rather than mutable task summary totals.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-12; BR-25; API-06.

Publish a release and inspect milestones

Primary actor: Project Manager. Preconditions: Project is ACTIVE; target release and referenced tasks belong to it.

Main success flow

01 Manager creates milestone with name and target date.

02 Manager links tasks and monitors milestone completion derivation.

03 Manager creates a PLANNED release with version label and target date.

04 Manager reviews included tasks and confirms Publish.

05 Server stores immutable release snapshot and marks state PUBLISHED.

06 Release summary displays included task keys and publication timestamp.

Alternative flow and error handling

Publishing without a valid version label fails; incomplete milestone cannot be force-completed by editing a status field.

Observable acceptance result

Later task edits do not rewrite already published release membership.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-26; API-03.

Inspect reports and historical metrics

Primary actor: Project Manager. Preconditions: Caller has access to project-level reporting and project contains activity history.

Main success flow

01 Manager selects sprint summary, cycle-time, workload or completion report.

02 Client sends project ID, report kind, filters and time range.

03 Server checks project membership and derives aggregate values from events.

04 Server returns chart-ready series and an as_of timestamp.

05 Client displays warning when data refresh is delayed beyond documented interval.

06 CSV export is requested separately under export authorization.

Alternative flow and error handling

Unknown report type returns INVALID_REPORT; reports from private projects must never contribute counts to other users.

Observable acceptance result

Report read does not mutate source tasks or recompute historical commitment snapshots.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-24; BR-29; API-14.

Archive a project and request restoration

Primary actor: Project Manager / Workspace Admin. Preconditions: Project has finished active development and archive action is authorized.

Main success flow

01 Project Manager chooses Archive and reviews read-only implications.

02 Server marks project ARCHIVED and freezes task/sprint mutations.

03 Members may continue to read historical project content.

04 Workspace ADMIN may request restore within documented recovery window.

05 Server checks archive timestamp and transitions project to ON_HOLD when eligible.

06 Project Manager explicitly resumes project to ACTIVE after restoration.

Alternative flow and error handling

Archive is reversible only during policy window; expired restore fails with RESTORE_WINDOW_EXPIRED.

Observable acceptance result

Activity keeps original project key and task IDs throughout archive and restoration.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-05; BR-29; API-03.

Subscribe and deliver a webhook

Primary actor: Workspace Admin / Integration. Preconditions: Admin configures a verified HTTPS endpoint and permitted event types.

Main success flow

01 Admin creates a webhook subscription with HTTPS URL and selected events.

02 Server validates destination restrictions and stores a hashed or protected secret reference.

03 A committed task update generates a durable outgoing delivery record.

04 Background worker signs payload and sends event ID and UTC timestamp.

05 Delivery outcome and next-attempt metadata become visible to admin.

06 Receiver may deduplicate retries using stable event ID.

Alternative flow and error handling

Invalid callback destinations fail validation; receiver timeouts do not roll back the original task change.

Observable acceptance result

No access token or short-lived attachment URL appears in webhook payload.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-30; ADR-04; API-15.

Preview and commit an import

Primary actor: Workspace Admin. Preconditions: Import bundle is a versioned CSV/JSON file and workspace is active.

Main success flow

01 Admin selects Import and uploads a structured data bundle.

02 Server validates schema and maps external IDs without persisting final objects.

03 Preview lists valid records, unknown assignees, state mapping errors and dependency cycles.

04 Admin corrects source file and repeats preview until required checks pass.

05 Admin commits preview token; server persists batch transactionally.

06 UI displays newly created keys and source-to-target ID mapping.

Alternative flow and error handling

An invalid preview cannot be committed; commit replay cannot create duplicate batch objects.

Observable acceptance result

Import source IDs remain traceable in import_reference and audit metadata.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-33; API-15.

Export project data with permissions

Primary actor: Project Manager / Workspace Admin. Preconditions: Caller may export selected project data and project belongs to workspace.

Main success flow

01 Caller opens project Export and selects JSON or CSV.

02 Server authorizes role and workspace/project scope.

03 Server creates an asynchronous export job with a UTC audit event.

04 Job produces task records, permitted comments and source document identifiers.

05 Caller receives a short-lived download link after job completes.

06 Link expires after 24 hours; job status remains visible without raw content.

Alternative flow and error handling

Guest exports and cross-workspace exports fail; downloading after expiry returns DOWNLOAD_EXPIRED.

Observable acceptance result

Export contents mirror authorization and preserve stable task IDs for future analysis.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-27; BR-29; API-15.

Handle permission loss while interacting

Primary actor: Contributor. Preconditions: User has a board tab open while a manager removes their project membership.

Main success flow

01 Contributor opens a task while their project membership is active.

02 Project Manager removes user’s project membership from a different session.

03 Contributor attempts to save a stale edit from the open tab.

04 Server re-evaluates current authorization and denies mutation.

05 Client discards cached protected content or redirects to accessible projects.

06 Audit records the role change, not an unauthorized attempted content mutation as a successful task change.

Alternative flow and error handling

Permission checks must not trust previously fetched board data or UI role state; denial does not reveal private project details.

Observable acceptance result

No unauthorized edit or cross-project data leak occurs after access revocation.

Review checklist

The user-facing flow must present the domain state returned by the server rather than assuming that a client-side operation succeeded.

An unauthorized, invalid or stale request must leave domain records unchanged, unless the flow explicitly documents an intermediate persisted state.

Relevant data changes must be visible in the audit or activity history required by the linked rules.

Relevant documents: BR-04; BR-06; BR-32; API-04.

Authentication and credentials

Versioned access-token and session endpoints. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/auth/register

Authorization: Public

Request: {"email":"dev@example.test","password":"<secret>"}

Success: 201 {"user_id":"usr_74","email_verified":false}

Errors: 400 INVALID_EMAIL; 409 EMAIL_ALREADY_REGISTERED.

Endpoint 2: POST /v1/auth/login

Authorization: Verified account

Request: {"email":"dev@example.test","password":"<secret>"}

Success: 200 {"access_token":"<redacted>","session_id":"ses_9","expires_in":900}

Errors: 401 INVALID_CREDENTIALS; 423 ACCOUNT_LOCKED.

Contract invariants

Refresh tokens are delivered using a secure channel, rotated on use and never logged. POST /v1/auth/logout revokes the current session and is safe to repeat. Date-time values are UTC.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-02; BR-03; FS-01; FS-02; ADR-05.

Workspace details and switching

Workspace selection and tenant-bound metadata. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: GET /v1/workspaces

Authorization: Signed-in user

Request: No body.

Success: 200 {"items":[{"id":"ws_1","name":"Design Org","role":"MEMBER"}]}

Errors: 401 UNAUTHENTICATED.

Endpoint 2: POST /v1/workspaces

Authorization: Signed-in verified user

Request: {"name":"Design Org"}

Success: 201 {"id":"ws_1","name":"Design Org","role":"OWNER"}

Errors: 400 INVALID_NAME; 429 RATE_LIMITED.

Contract invariants

Client must provide workspace context for nested operations, while server resolves membership from its own datastore. An ID alone confers no access to another tenant.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-01; BR-04; FS-03.

Projects and lifecycle

Create, inspect and change project state. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/workspaces/{ws}/projects

Authorization: ADMIN or OWNER

Request: {"key":"MOB","name":"Mobile Client","timezone":"Europe/Bucharest"}

Success: 201 {"id":"prj_9","key":"MOB","state":"DRAFT"}

Errors: 400 INVALID_PROJECT_KEY; 409 PROJECT_KEY_EXISTS.

Endpoint 2: PATCH /v1/projects/{project}/state

Authorization: Project Manager

Request: {"state":"ACTIVE","reason":"Kickoff complete"}

Success: 200 {"id":"prj_9","state":"ACTIVE","version":2}

Errors: 409 INVALID_PROJECT_TRANSITION; 403 FORBIDDEN.

Contract invariants

Conditional requests use resource version where applicable. Archived project operations remain readable but reject mutating methods. API keys are opaque identifiers.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-05; BR-26; BR-29; FS-04; FS-20.

Invitations and membership

Workspace and project grants. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/workspaces/{ws}/invites

Authorization: Workspace ADMIN

Request: {"email":"alex@example.test","role":"MEMBER","project_ids":["prj_9"]}

Success: 201 {"invite_id":"inv_7","expires_in_days":7}

Errors: 403 ROLE_NOT_ALLOWED; 409 MEMBER_ALREADY_EXISTS.

Endpoint 2: POST /v1/projects/{project}/members

Authorization: Project Manager

Request: {"user_id":"usr_74","project_role":"CONTRIBUTOR"}

Success: 201 {"member_id":"pm_12","project_role":"CONTRIBUTOR"}

Errors: 403 FORBIDDEN; 404 USER_NOT_IN_WORKSPACE.

Contract invariants

Membership list endpoint applies a project capacity rule of at most 100 active members; excess returns PROJECT_MEMBER_LIMIT. Role changes must not be taken from client-supplied claims.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-02; BR-04; BR-06; FS-03; FS-05.

Project reads and member search

Project detail, visibility and eligible assignees. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: GET /v1/projects/{project}

Authorization: Authorized project member

Request: No body.

Success: 200 {"key":"MOB","state":"ACTIVE","visibility":"RESTRICTED"}

Errors: 403 FORBIDDEN; 404 PROJECT_NOT_FOUND.

Endpoint 2: GET /v1/projects/{project}/members?eligible_for=assignee

Authorization: Project member

Request: No body.

Success: 200 {"items":[{"user_id":"usr_74","role":"CONTRIBUTOR"}]}

Errors: 400 INVALID_FILTER; 404 PROJECT_NOT_FOUND.

Contract invariants

The server filters membership metadata to caller visibility. A workspace MEMBER does not automatically gain access to a private project; guests see only minimal collaborator identities.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-04; BR-06; BR-09; FS-05.

Task create and update

Primary task identity and editable attributes. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/projects/{project}/tasks

Authorization: CONTRIBUTOR or Project Manager

Request: {"type":"STORY","title":"Add new modal","description":"...","priority":"P2"}

Success: 201 {"id":"task_42","key":"MOB-0042","state":"BACKLOG"}

Errors: 400 VALIDATION_FAILED; 403 FORBIDDEN; 409 PROJECT_ARCHIVED.

Endpoint 2: PATCH /v1/tasks/{task}

Authorization: Authorized editor

Request: {"title":"Add accessible modal","estimate_minutes":240,"version":3}

Success: 200 {"key":"MOB-0042","version":4}

Errors: 409 VERSION_CONFLICT; 400 INVALID_FIELD.

Contract invariants

Story criteria validation applies before READY, and estimated versus actual minutes are separate fields. Task keys remain constant despite title edits or report recalculation.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-07; BR-12; BR-14; FS-06; FS-18.

Task transitions and reopen

Controlled task state mutation. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/tasks/{task}/transitions

Authorization: Eligible project contributor

Request: {"target_state":"IN_REVIEW","reason":"Ready for review"}

Success: 200 {"key":"MOB-0042","state":"IN_REVIEW"}

Errors: 409 INVALID_TRANSITION; 409 BLOCKING_TASK_OPEN.

Endpoint 2: POST /v1/tasks/{task}/reopen

Authorization: Project Manager

Request: {"reason":"Regression found"}

Success: 200 {"key":"MOB-0042","state":"IN_PROGRESS"}

Errors: 403 FORBIDDEN; 400 REASON_REQUIRED.

Contract invariants

Server determines transitions from current persisted state. Reopen is recorded as its own audit event. A stale UI cannot bypass blocker or subtask completion checks.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-08; BR-11; BR-13; FS-07.

Task assignment and claim

One primary assignee per task. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: PUT /v1/tasks/{task}/assignee

Authorization: Project Manager

Request: {"user_id":"usr_74"}

Success: 200 {"key":"MOB-0042","assignee_id":"usr_74"}

Errors: 400 ASSIGNEE_NOT_ELIGIBLE; 403 FORBIDDEN.

Endpoint 2: POST /v1/tasks/{task}/claim

Authorization: CONTRIBUTOR

Request: {}

Success: 200 {"key":"MOB-0042","assignee_id":"usr_self"}

Errors: 409 TASK_ALREADY_ASSIGNED; 403 FORBIDDEN.

Contract invariants

Assignee changes audit old and new user IDs. Both endpoints reject archived tasks and enforce tenant/project access at the moment of mutation.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-09; BR-21; FS-08.

Dependencies and subtasks

Graph links and one-level decomposition. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/tasks/{task}/dependencies

Authorization: Authorized contributor

Request: {"target_task_key":"MOB-0012","type":"BLOCKS"}

Success: 201 {"source":"MOB-0042","target":"MOB-0012"}

Errors: 409 DEPENDENCY_CYCLE; 400 INVALID_TARGET.

Endpoint 2: POST /v1/tasks/{task}/subtasks

Authorization: Authorized contributor

Request: {"title":"Build API serializer","type":"TASK"}

Success: 201 {"key":"MOB-0043","parent_key":"MOB-0042"}

Errors: 400 INVALID_PARENT; 403 FORBIDDEN.

Contract invariants

Cross-tenant existence is concealed. Cycle detection is transactional, including concurrent proposals, and DONE transition checks evaluate blocker state at commit time.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-11; BR-13; FS-09.

Sprint planning and activation

Create sprints and commit planning baseline. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/projects/{project}/sprints

Authorization: Project Manager

Request: {"name":"Sprint 12","start":"2026-11-02","end":"2026-11-16"}

Success: 201 {"id":"spr_12","state":"PLANNED"}

Errors: 400 INVALID_SPRINT_DATES; 409 DATE_OVERLAP.

Endpoint 2: POST /v1/sprints/{sprint}/start

Authorization: Project Manager

Request: {"expected_version":4}

Success: 200 {"id":"spr_12","state":"ACTIVE","commitment_count":18}

Errors: 409 ACTIVE_SPRINT_EXISTS; 409 VERSION_CONFLICT.

Contract invariants

Dates use project time zone. Start operation writes immutable baseline task IDs and estimates transactionally, after validating a single active sprint per project.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-17; BR-18; FS-10; FS-11.

Sprint tasks and completion

Link tasks and complete execution. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/sprints/{sprint}/tasks

Authorization: Project Manager

Request: {"task_keys":["MOB-0042","MOB-0043"]}

Success: 200 {"added":2,"scope_change":true}

Errors: 409 TASK_IN_OTHER_SPRINT; 403 FORBIDDEN.

Endpoint 2: POST /v1/sprints/{sprint}/complete

Authorization: Project Manager

Request: {"summary":"Sprint goal partially met"}

Success: 200 {"state":"COMPLETED","done":12,"unfinished":4}

Errors: 409 SPRINT_NOT_ACTIVE; 403 FORBIDDEN.

Contract invariants

Completion retains the commitment snapshot and creates a report event. Repeated completion with an idempotency key must not move unfinished tasks twice.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-18; BR-19; FS-12.

Comments and file attachments

Task conversation and binary attachment metadata. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/tasks/{task}/comments

Authorization: CONTRIBUTOR

Request: {"body":"Please review @alex","mentioned_user_ids":["usr_74"]}

Success: 201 {"id":"com_8","version":1}

Errors: 400 BODY_TOO_LONG; 403 FORBIDDEN.

Endpoint 2: POST /v1/tasks/{task}/attachments

Authorization: CONTRIBUTOR

Request: {"filename":"screen.png","size_bytes":25000000,"mime":"image/png"}

Success: 201 {"id":"att_5","scan_status":"PENDING"}

Errors: 413 FILE_TOO_LARGE when size exceeds 25 MiB; 415 MIME_NOT_ALLOWED.

Contract invariants

GET /v1/attachments/{id}/download requires renewed authorization. Sanitized comment Markdown never executes scripts; the file remains in quarantine until scan passes.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-15; BR-16; FS-13; FS-14.

Notifications and preferences

Inbox reads and channel configuration. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: GET /v1/me/notifications?unread=true

Authorization: Authenticated user

Request: No body.

Success: 200 {"items":[{"id":"ntf_8","event":"task.assigned","read":false}]}

Errors: 401 UNAUTHENTICATED.

Endpoint 2: PATCH /v1/me/notification-preferences

Authorization: Authenticated user

Request: {"event":"task.mentioned","channel":"EMAIL","enabled":true}

Success: 200 {"event":"task.mentioned","channel":"EMAIL","enabled":true}

Errors: 400 INVALID_EVENT; 400 INVALID_CHANNEL.

Contract invariants

Notification records deduplicate by event ID plus recipient ID and expose only content the reader may currently access. Preference writes do not retroactively send old events.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-21; FS-17.

Search and reports

Permission-aware query and metric APIs. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: GET /v1/search/tasks?q=login&project=MOB

Authorization: Workspace member

Request: Query parameters: q, project, state, cursor.

Success: 200 {"items":[{"key":"MOB-0042","title":"Login bug"}],"next_cursor":null}

Errors: 400 INVALID_FILTER; 404 PROJECT_NOT_FOUND.

Endpoint 2: GET /v1/projects/{project}/reports/sprint-summary

Authorization: Project Manager

Request: Query: from=2026-10-01&to=2026-11-01

Success: 200 {"done":12,"committed":16,"as_of":"2026-10-27T12:00:00Z"}

Errors: 403 FORBIDDEN; 400 INVALID_REPORT.

Contract invariants

Report aggregate cache refresh interval is 60 minutes. Search applies visibility filters before producing facets and result counts; dates and cursors are opaque client inputs.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-23; BR-24; FS-16; FS-20.

Exports, imports and integration hooks

Long-running jobs and outbound notifications. Endpoints below use /v1, JSON payloads, authenticated principals where stated and resource IDs that are opaque strings.

Endpoint 1: POST /v1/projects/{project}/exports

Authorization: Workspace ADMIN or OWNER

Request: {"format":"CSV","include_comments":true}

Success: 202 {"job_id":"job_9","status":"PENDING"}

Errors: 403 FORBIDDEN; 400 INVALID_FORMAT.

Endpoint 2: POST /v1/workspaces/{ws}/webhooks

Authorization: Workspace ADMIN

Request: {"url":"https://example.test/hook","events":["task.updated"]}

Success: 201 {"webhook_id":"wh_3","status":"ACTIVE"}

Errors: 400 INVALID_CALLBACK_URL; 403 FORBIDDEN.

Contract invariants

Import previews and commits are exposed as POST /v1/workspaces/{ws}/imports/preview and /commit with a preview token. Job status endpoints return state without leaking payload before authorization.

All errors use {"error":{"code":"UPPER_SNAKE_CASE","message":"...","correlation_id":"..."}}. Authorization is evaluated using current workspace and project grants.

Client-supplied resource IDs never bypass tenant scoping. All timestamps are UTC ISO 8601; date-only values follow the project time zone.

Related policy and flows: BR-29; BR-30; BR-33; FS-22; FS-23; FS-24.

Modular monolith for initial release

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

TeamFlow needs many closely related transactions across projects, tasks, sprints and permissions.

Decision

Deploy one server application with modules for identity, workspace, projects, tasks, planning, notifications and reporting. Cross-module references use explicit internal interfaces.

Alternative considered

Microservices per domain were considered, but distributed authorization and consistency would exceed first-release operational needs.

Benefits and rationale

Simplifies shared transactions, local development and cross-domain rule enforcement.

Costs and trade-offs

Module boundaries can erode unless reviewed; deployment still scales as a unit.

Verification criteria

No public handler reaches directly into another module’s persistence tables without a designated interface.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

PostgreSQL as transactional system of record

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Tenant-scoped projects, task transitions, membership and sprint constraints require transactional data integrity.

Decision

Use PostgreSQL to store relational domain records, unique keys, membership grants and audit references. Enforce foreign keys and unique constraints.

Alternative considered

A document database was considered for flexibility, but would make dependency and membership consistency harder to audit.

Benefits and rationale

Transactions guard task state checks and writes; relational constraints protect stable keys.

Costs and trade-offs

High-volume full-text workloads need a separate index and deliberate refresh strategy.

Verification criteria

Tests attempt concurrent creation of the same project key, conflicting sprint activations and cyclic dependency links.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Tenant separation through workspace IDs

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

An account may work in multiple organizations while every API and background job must honor tenant boundaries.

Decision

Each domain row includes workspace_id; repository calls require trusted workspace context and access policies; exports recheck scope.

Alternative considered

Database-per-tenant was considered but rejected for first-release administration and migration complexity.

Benefits and rationale

Consistent scoping applies to REST endpoints, report generation, search and file access.

Costs and trade-offs

Missed scoping can cause severe leakage; code review and automated isolation tests are mandatory.

Verification criteria

Cross-tenant object ID probes always return nonrevealing responses, including for search facets and downloadable files.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Event-driven notifications and webhooks

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Task mutations must not block on email services or third-party webhook availability.

Decision

Write an outbox event with a successful domain transaction, then process notifications and webhooks asynchronously via retryable workers.

Alternative considered

Synchronous HTTP calls inside task transactions were rejected because remote failures would make local tasks unreliable.

Benefits and rationale

External failures cannot roll back successful updates; event IDs support deduplication.

Costs and trade-offs

Delivery is eventually consistent, requiring visible retry status and safe dead-letter handling.

Verification criteria

Duplicate event delivery causes no duplicate in-app notifications or other internal side effects.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Short-lived access tokens and rotating sessions

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

The browser client needs secure access to workspace APIs with immediate revocation when membership or session security changes.

Decision

Issue short-lived access tokens with a server-backed session and refresh-token rotation; all API handlers perform current authorization.

Alternative considered

Long-lived bearer tokens without session state were rejected because revocation would lag.

Benefits and rationale

Sessions can be revoked and token compromise detected through refresh reuse.

Costs and trade-offs

Interactive user experience needs token refresh handling and session cleanup.

Verification criteria

Login lockout, logout, refresh reuse and role revocation have integration tests.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Object storage with quarantine scanning

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Attachments may contain untrusted binary data and task visibility can change after upload.

Decision

Keep file bytes in private object storage; scan before release; issue time-limited download URLs after server-side authorization.

Alternative considered

Storing blobs directly in the primary relational database was rejected to simplify capacity and virus-scanning workflows.

Benefits and rationale

Preserves task database performance and supports separate file retention controls.

Costs and trade-offs

Object lifecycle reconciliation must clean up abandoned quarantined uploads.

Verification criteria

No file in PENDING or REJECTED scan status is retrievable through the application or direct object URL.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Versioned REST API contracts

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

The browser and integration clients need stable contracts while business rules evolve.

Decision

Publish /v1 endpoints, consistent error envelopes, opaque IDs, optimistic versions and documented idempotency semantics.

Alternative considered

Unversioned mutable endpoints were rejected because they would break dependent clients silently.

Benefits and rationale

Allows contract tests and additive changes within the same major version.

Costs and trade-offs

Major changes require migration, deprecation notice and compatibility validation.

Verification criteria

Each endpoint has machine-testable schema and a rejection case for unauthorized access.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Search index derived from source records

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Full-text search must be responsive without becoming an authority for task state or access control.

Decision

Build asynchronous search projections from committed task and comment events; filter results by current database-derived authorization.

Alternative considered

Searching solely via SQL LIKE across all comments was rejected for expected scale and relevance limitations.

Benefits and rationale

Flexible text search with manageable transactional coupling.

Costs and trade-offs

Index lag requires defined UX behavior and permission rechecks before result display.

Verification criteria

Deleting a project membership immediately hides its tasks from search even if index deletion is delayed.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Append-only audit trail for sensitive mutations

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Reopen, reassignment and permission changes must be attributable for debugging and evaluating ticket specifications.

Decision

Record immutable audit events with actor, workspace, entity, before/after references, UTC time and correlation ID.

Alternative considered

Overwriting one latest_activity field was rejected because it loses historical evidence.

Benefits and rationale

Offers reproducible change history and explicit links among records.

Costs and trade-offs

Audit volume and personal-data retention require bounded storage and access controls.

Verification criteria

No endpoint accepts a client-supplied actor as authoritative; audit exports honor tenant boundaries.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Operational backup and recovery

Status: Accepted for the fictional TeamFlow baseline. The decision governs implementation until replaced by a documented ADR.

Context and problem

Accidental deletion, database failure and file-store incidents must not silently invalidate active projects.

Decision

Create encrypted daily database backups and keep them for 30 days; object metadata recovery is checked with periodic restore rehearsals.

Alternative considered

Relying on replicas alone was rejected because replicas reproduce logical deletion and corruption.

Benefits and rationale

Provides a documented disaster-recovery path and auditable rehearsal evidence.

Costs and trade-offs

Backup storage must be isolated; recovery may need manual reconciliation of queued integrations.

Verification criteria

A quarterly staging restore demonstrates referential integrity and successful access to authorized project artifacts.

Change control

A proposal to supersede this decision must identify impacted business-rule IDs, changed API contracts, migration implications and the result of the verification check above. This ADR does not permit bypassing tenant isolation, permission checks or required audit events.

Operational references: BR-01; BR-22; BR-32; GL-07; GL-08.

Naming, IDs and schema conventions

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-01-R01 Resource IDs are opaque prefixes: ws_, usr_, prj_, task_, spr_, com_, att_, ntf_ and job_.

GL-01-R02 Human-facing task keys use PROJECTKEY-NNNN and remain immutable after creation.

GL-01-R03 Database field and API JSON keys use snake_case; domain enums use UPPER_SNAKE_CASE.

GL-01-R04 Plural REST resource names represent collections; verbs appear only on explicit transition actions.

GL-01-R05 Each schema change has a migration and rollback strategy; never reuse a retired enum code.

GL-01-R06 Document headings carry stable BR, FS, API, ADR or GL IDs and do not depend on pagination.

Concrete example

Example valid task key: MOB-0042; valid JSON: {"assignee_id":"usr_74","state":"IN_REVIEW"}.

Review and test criterion

CI checks schema examples and IDs for uniqueness.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Error envelopes and client feedback

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-02-R01 All non-2xx API responses use an error object with code, message and correlation_id.

GL-02-R02 Codes such as INVALID_TRANSITION, NOT_FOUND and FORBIDDEN are stable client-facing identifiers.

GL-02-R03 Validation responses include field_errors array without echoing secrets.

GL-02-R04 400 means invalid submitted shape; 401 requires authentication; 403 denies an authorized identity; 409 signals state conflict.

GL-02-R05 Unauthorized cross-tenant resource lookups may respond 404 to avoid leaking existence.

GL-02-R06 Client error messages identify the actionable field and never show internal stack traces.

Concrete example

Example: {"error":{"code":"INVALID_TRANSITION","message":"Use an allowed state transition","correlation_id":"cor_2"}}.

Review and test criterion

Contract tests assert status, code and privacy behavior.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

05 / GUIDELINES / CONVENTIONS · GL-03 · SOURCE PAGE 088/100

Time zones, dates and numeric units

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-03-R01 All event and audit timestamps are ISO 8601 UTC strings with trailing Z.

GL-03-R02 User calendar dates are YYYY-MM-DD; project-local scheduling uses stored IANA time zone.

GL-03-R03 Sprint start is inclusive and end is exclusive; duration is computed in calendar days.

GL-03-R04 Time estimates and actual effort are integer minutes, not floating hours.

GL-03-R05 Ratios in reporting are returned as raw numerator/denominator plus derived percentages.

GL-03-R06 Formatting in Romanian, English or other locale never changes API representation.

Concrete example

Example: completed_at="2026-10-27T12:01:15Z", due_date="2026-10-28", timezone="Europe/Bucharest".

Review and test criterion

Tests cover daylight-saving transitions and midnight boundaries.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Pagination, filtering and sorting

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-04-R01 List endpoints default to limit 25 and accept limit from 1 through 100.

GL-04-R02 Cursor pagination is preferred where records can be inserted during navigation.

GL-04-R03 Every response returns items and next_cursor, with null for the last page.

GL-04-R04 Filters are explicit known field names; arbitrary expressions and free-form SQL are prohibited.

GL-04-R05 Sorts are restricted to documented keys with deterministic ID tie-breaking.

GL-04-R06 Each page of results rechecks current workspace and project authorization.

Concrete example

Example: GET /v1/search/tasks?project=MOB&limit=25&cursor=cur_89.

Review and test criterion

Integration tests insert records between pages and assert no cross-tenant entries.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Idempotency, concurrency and retries

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-05-R01 Clients send Idempotency-Key for endpoints explicitly marked retry-safe for creation.

GL-05-R02 An idempotency key is scoped by workspace, principal and operation; payload mismatch returns 409.

GL-05-R03 Updates involving race conditions use optimistic resource versions and reject stale writers.

GL-05-R04 Outbox and webhook workers deduplicate stable event IDs before irreversible side effects.

GL-05-R05 Never retry unsafe mutations automatically without idempotency semantics.

GL-05-R06 429 and transient 503 responses may include Retry-After and are handled with bounded backoff.

Concrete example

Example: repeating one export creation request with identical key returns the original job_id.

Review and test criterion

Concurrency tests try two sprint activations and simultaneous dependency cycles.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Security, role checks and session handling

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-06-R01 All API operations validate live workspace and project grants server-side.

GL-06-R02 Access tokens must not be included in logs, URLs, exports or webhook bodies.

GL-06-R03 Every sensitive state mutation records actor and correlation ID.

GL-06-R04 Browser access is protected using secure transport and anti-cross-site scripting rules.

GL-06-R05 Session inactivity threshold is 45 minutes before reauthentication is required.

GL-06-R06 Admin privileges do not bypass membership checks in unrelated tenants.

Concrete example

Example: stale browser tab after project removal cannot save an edit and receives a safe 403/404 response.

Review and test criterion

Security tests probe scope bypass, token reuse and suspicious redirect paths.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Test cases and acceptance-criteria mapping

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-07-R01 Each user story must link acceptance criteria to relevant business rules with exact IDs.

GL-07-R02 Write positive, invalid-input, wrong-role and stale-version tests for each mutating API.

GL-07-R03 Test fixtures use fictional accounts and domains such as example.test, not real credentials.

GL-07-R04 When a document has an unresolved decision, do not silently encode a default in tests.

GL-07-R05 Contract examples in the API section are checked against the implementation schema.

GL-07-R06 Never use golden evaluation labels as part of the agent's inference prompt.

Concrete example

Example: test TF-TRANSITION-04 links BR-08-R06 and expects INVALID_TRANSITION for BACKLOG-to-DONE.

Review and test criterion

Each regression test identifies its related BR or FS source.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Logs, tracing and observability

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-08-R01 Generate a correlation_id for each inbound request and propagate it into asynchronous jobs.

GL-08-R02 Log timestamp, severity, component, operation code and elapsed time.

GL-08-R03 Never log passwords, refresh tokens, webhook shared secrets or attachment bytes.

GL-08-R04 Background jobs expose retries, current state and last safe error category.

GL-08-R05 Errors from integrations use a sanitized message and structured internal code.

GL-08-R06 Metrics may group counts by authorized workspace ID but public dashboards omit private names.

Concrete example

Example audit reference: request cor_120 led to task.updated event evt_150 and notification ntf_12.

Review and test criterion

Operations review a sample failed request without needing customer secrets.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Accessibility and user-facing language

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-09-R01 All essential flows work with keyboard, screen-reader labels and visible focus.

GL-09-R02 Form errors refer to the affected input and explain a corrective action.

GL-09-R03 Color cannot be the sole indication of priority, error or task state.

GL-09-R04 Domain enums remain invariant across locale while UI strings are translatable.

GL-09-R05 Confirmation dialogs identify irreversible actions, such as completed sprint close.

GL-09-R06 Date and number rendering follows user locale but exported records retain canonical types.

Concrete example

Example board transition is available from a task context menu without requiring drag-and-drop.

Review and test criterion

QA includes automated accessibility checks plus manual keyboard traversal.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

Documentation governance and references

Engineering conventions apply to TeamFlow services, clients, documentation and integration tests. Conventions do not replace normative business rules.

Required engineering conventions

GL-10-R01 Every substantive source section has an immutable ID and a status: Draft, Approved or Deprecated.

GL-10-R02 Changes to normative rules include author role, date, reason and referenced tickets.

GL-10-R03 An example never silently overrides a conflicting normative rule.

GL-10-R04 When documents disagree, engineering must request an explicit decision; do not assume newest means correct.

GL-10-R05 Ticket clarification notes must cite the exact approved requirement IDs and explicit open decisions.

GL-10-R06 Client-facing docs and API schemas must not expose curator-only labels or synthetic evaluation categories.

Concrete example

Example: if BR-18-R04 and FS-12 describe different rollover behavior, submit both evidence IDs to the product owner for an explicit decision.

Review and test criterion

Document review checks stable IDs and cross-references before dataset generation.

Documentation practice

Use exact source IDs when discussing requirement changes. If two approved sources disagree, preserve their statements until a product decision is recorded; do not silently treat an example, API response or an undocumented assumption as an overriding policy.

Normative domain definitions are primarily located in BR-01 through BR-35 and the relevant API contracts.

06 / FAQ / TROUBLESHOOTING · FAQ-01 · SOURCE PAGE 096/100

Accounts, invitations and workspace access

Troubleshooting answers describe observable TeamFlow behavior for support and engineering teams. Source references are provided with each answer for verification.

Q1. Why can a verified user see no projects?

Verification creates an account, not workspace or project membership. Accept an invitation and ask a project manager for the project grant.

References: BR-02; BR-04; FS-03.

Q2. Why does an invitation link return EXPIRED?

Invitations are single-use and expire after seven calendar days; an admin must issue a fresh one.

References: BR-02; API-04.

Q3. Can a workspace MEMBER view every project?

No. Project membership is explicit. Workspace administrators and owners have administrative access, subject to tenant boundaries.

References: BR-04; BR-06.

Q4. Why did a previously visible task disappear?

Project membership may have been removed; the server checks current authorization on every read and mutation.

References: BR-06; FS-25.

Q5. Can the last workspace OWNER leave?

No. Another OWNER must already exist before the last owner role may be removed.

References: BR-04.

Q6. Why was my account temporarily locked?

Five consecutive incorrect password attempts trigger a fifteen-minute lockout. Reset or wait according to account policy.

References: BR-03; FS-02.

Escalation guidance

If a ticket raises behavior not explicitly answered by these sources, request the missing product decision rather than inventing a default. If this FAQ disagrees with another document, cite both exact passages for a product-owner clarification.

06 / FAQ / TROUBLESHOOTING · FAQ-02 · SOURCE PAGE 097/100

Tasks, subtasks and comments

Troubleshooting answers describe observable TeamFlow behavior for support and engineering teams. Source references are provided with each answer for verification.

Q1. Can a task jump directly from BACKLOG to DONE?

No. It must use the authorized state transitions through READY, IN_PROGRESS and IN_REVIEW.

References: BR-08; FS-07.

Q2. Why is Done disabled for my task?

An unresolved BLOCKS dependency or unfinished subtask can prevent completion even when review passes.

References: BR-11; BR-13.

Q3. Who may reopen a DONE task?

Only a Project Manager with a nonempty reason can reopen DONE to IN_PROGRESS.

References: BR-08; API-07.

Q4. How long may authors edit comments?

Authors may edit their own comments for 60 minutes; managers can redact for policy reasons.

References: BR-15; FS-13.

Q5. What happens if a contributor is removed?

Open-task assignee references are cleared while old activity and authored comments remain attributable.

References: BR-09; BR-31.

Q6. Can subtasks have subtasks?

No. Only one subtask level is supported and parent and child must be in the same project.

References: BR-13.

Escalation guidance

If a ticket raises behavior not explicitly answered by these sources, request the missing product decision rather than inventing a default. If this FAQ disagrees with another document, cite both exact passages for a product-owner clarification.

06 / FAQ / TROUBLESHOOTING · FAQ-03 · SOURCE PAGE 098/100

Notifications, preferences and audit

Troubleshooting answers describe observable TeamFlow behavior for support and engineering teams. Source references are provided with each answer for verification.

Q1. Will mentioning someone in a private project notify them?

Only if the mentioned user currently has permission to read the task.

References: BR-15; BR-21.

Q2. Are mention emails turned on for a new account?

Yes. Mention email delivery is enabled initially and can be disabled in notification preferences.

References: BR-21; FS-17.

Q3. Why did I receive no email but see an in-app alert?

Notification channels are independently configurable, and queued email may fail or be disabled.

References: BR-21; ADR-04.

Q4. Can administrators read the task audit history?

Authorized workspace admins can inspect workspace audit views, while task-level history requires suitable project visibility.

References: BR-22.

Q5. Do repeated webhook deliveries create duplicate events?

Receivers should deduplicate by stable event ID; internal in-app notifications also deduplicate by event and recipient.

References: BR-30; ADR-04.

Q6. Can a guest export task comments?

No. Guests may read permitted comments but cannot run project exports.

References: BR-27; BR-29.

Escalation guidance

If a ticket raises behavior not explicitly answered by these sources, request the missing product decision rather than inventing a default. If this FAQ disagrees with another document, cite both exact passages for a product-owner clarification.

Projects, sprints and exports

Troubleshooting answers describe observable TeamFlow behavior for support and engineering teams. Source references are provided with each answer for verification.

Q1. Why can I not start the planned sprint?

A project may have only one ACTIVE sprint; complete the current sprint first.

References: BR-18; FS-11.

Q2. What happens to unfinished sprint tasks?

Sprints preserve historical commitment. Incomplete tasks are handled by the configured rollover behavior described in planning specifications.

References: BR-18; FS-12.

Q3. Can an archived project be restored after one month?

Workspace administrators may restore an archived project within 30 days of archiving.

References: BR-29; FS-21.

Q4. Why is an archived project read-only?

Archiving freezes task and sprint mutations to preserve historical project context.

References: BR-05; BR-29.

Q5. Can a project manager export their own project?

Yes, project export actions are available to managers with access and eligible workspace admins.

References: BR-29.

Q6. What does the export include?

Task records, allowed comment metadata and identifiers, with authorized scope enforced and links expiring after 24 hours.

References: BR-29; FS-24.

Escalation guidance

If a ticket raises behavior not explicitly answered by these sources, request the missing product decision rather than inventing a default. If this FAQ disagrees with another document, cite both exact passages for a product-owner clarification.

06 / FAQ / TROUBLESHOOTING · FAQ-05 · SOURCE PAGE 100/100

Files, search, reports and limits

Troubleshooting answers describe observable TeamFlow behavior for support and engineering teams. Source references are provided with each answer for verification.

Q1. Why is my attachment not yet downloadable?

Files remain quarantined until scanning completes successfully.

References: BR-16; ADR-06.

Q2. What is the maximum upload size?

The documented business limit is 20 MiB per task attachment; unsupported MIME types are rejected.

References: BR-16.

Q3. Can a search result reveal an inaccessible project?

No. Authorization is applied to search matches, counts and snippets before rendering.

References: BR-23; ADR-08.

Q4. How fresh is the cycle-time dashboard?

Each report carries as_of. Normal aggregate refresh is at least every fifteen minutes.

References: BR-24.

Q5. Why was creating another project denied?

Your workspace plan may have reached its active-project quota; downgrades do not delete existing projects.

References: BR-35.

Q6. Does task priority automatically change sprint allocation?

No. Priority and sprint placement are independent; planning must be explicitly updated by an authorized manager.

References: BR-10; BR-19.

Escalation guidance

If a ticket raises behavior not explicitly answered by these sources, request the missing product decision rather than inventing a default. If this FAQ disagrees with another document, cite both exact passages for a product-owner clarification.